

Autonomous SDR - 40-Hour Buildathon Plan

Inter Guild Buildathon 2026 · Team of 3 · Selling for Sarvam AI · Built with Claude Code + DronaHQ

1. The Core Idea

Most teams will build seven agents that each fire messages on their own schedule. That looks like “five disconnected bots,” and judges will notice. Your system should have **one brain per prospect**.

One owner per prospect. Each prospect in a campaign has one state record and one unified timeline across every channel. The **Outreach Strategy Agent** is the single decision-maker for that prospect. It looks at the whole timeline and picks the next best action: channel, timing, and intent. The other agents are specialists it calls - Personalisation writes, Voice calls, Conversation interprets replies, Follow-up handles timing.

“The LLM proposes, the code disposes.” Agents suggest actions. A deterministic **policy gate**, written in code, decides whether each action is allowed. It runs right before every external action, not only when the action is queued. It checks:

- kill switch, then campaign status, then agent pause, then channel pause
- suppression list, then global contact-frequency cap
- rep working hours and daily limits
- approval requirements

This one function gives four things: pause that is instant even mid-execution, conflict handling between campaigns, safety, and a strong engineering story for judges.

Every action carries its “why.” Each agent run stores its inputs, the knowledge-base chunks it retrieved, the prompt version that was active, a one-line reason in plain English, and its tokens, cost, and latency. The UI shows the reason by default and hides the details behind a click. This answers the brief’s “why did the agent behave this way?” requirement and makes the product readable at a glance.

2. What to Sell: Sarvam AI

DronaHQ remains **mandatory as the platform** (15 rubric points) - it stays the control-plane UI plus a couple of hosted agents. What changes is **the company your SDR sells for**: an Indian startup instead of DronaHQ itself.

Sarvam AI is the strongest fit for demonstrating every use case:

- **Real Indian BFSI buyers.** Sarvam’s stated focus verticals are banking, insurance, gov-tech, and defence, and its conversational platform reportedly handles over 2 million interactions a day. That supports a credible campaign aimed at Indian bank and insurer CIOs.
- **Natural buyers among voice-AI founders.** Sarvam sells developer products - Saaras (speech-to-text across all 22 scheduled Indian languages) and Bulbul (text-to-speech) - and runs a startup program offering 6–12 months of API credits plus engineering support. Founders building voice products are an obvious ICP with an easy call to action (“apply for credits”).
- **Enough product surface for separate campaigns.** Sarvam’s Indus platform bundles Samvaad (voice agents), Work, Code, Content Agents, DocAgent, and India-hosted inference. Each campaign can sell a different product to a different buyer.
- **Plenty of public material** - blog posts, docs, and press coverage - for a real knowledge base.

You also get a strong demo moment: as an IIT Madras team, your Voice SDR can place its call in Hindi or Tamil.

Disclaimer required in the report and the UI. You are not affiliated with Sarvam. The knowledge base comes only from public information. All outreach goes to contacts your team controls. Show a small “Demo - not affiliated with Sarvam AI” footer.

The four campaigns

	A: BFSI Voice Agents (India)	B: Voice AI Founders (India)	C: US SaaS Expanding to India	D: Existing Customer Expansion
ICP	CIO / CDO / Head of CX at Indian banks, NBFCs, insurers	Founders/CTOs of seed-Series A Indian voice/conversational AI startups	CTO / VP Eng at US SaaS companies with Indian users	Current API customers (CSV as CRM)
Sells	Samvaad voice agents for collections, KYC, claims; India-hosted data	Speech APIs + startup program credits	Indic-language voice/text support inside their product	Upgrade to enterprise plan / voice-agent platform
Tone	Formal, compliance-aware	Casual, founder-to-founder	Crisp, US business tone	Warm, relationship-based
Channels	LinkedIn → email → call only after engagement	WhatsApp + email + quick call	Email + LinkedIn only, no cold calls	Email from the assigned rep
Policy	Approval on every first touch; slow cadence; pricing escalates	Fully autonomous; fast cadence	US working hours only (timezone logic)	Uses rep’s identity/signature
Proves	Human-in-the-loop, escalation, conservative policy	Full autonomous multi-channel loop, voice booking	Geography, timezones, channel restrictions	Draft → checklist → Live live in the demo; rep assignment

Experimentation stretch: duplicate Campaign B into **B-Variant** with a different personalisation angle (“lead with a pain point” vs. “lead with a credits offer”) and compare on the Compare page.

Planned demo scenarios (build these into the data on purpose)

- **Conflict** - one founder appears in both Campaign B and Campaign D.
- **Objection** - a prospect replies “we already use another speech vendor.”
- **Escalation** - a pricing question; the guardrail blocks invented pricing and escalates to a human.
- **Suppression** - a reply of “unsubscribe” suppresses the prospect across all campaigns.
- **Out-of-office** - an auto-reply triggers a delayed follow-up.
- **Wrong person** - a reply of “talk to my colleague” creates a referral prospect.

3. How the Work Splits (3 people)

Person	Owns	Hands off to the others
Product (P)	DronaHQ control plane, knowledge-base content, prospect/persona data, golden-dataset labels, visual design, demo script, report, deck, README, QA	Screens that call Software's API; KB documents for AI; labelled data
AI	Every agent (prompts, output schemas, retrieval, guardrails), the LLM wrapper, embeddings, voice-agent setup, DronaHQ-hosted agents, evals, cost tracking	A Python package <code>agents/</code> with plain functions Software can call
Software (S)	Database, backend API, scheduler/job runner, policy gate, channel integrations (email, WhatsApp, voice, calendar, LinkedIn simulator), webhooks, deployment, security	A deployed API that Product's screens consume

Two agreements that keep 3 people from blocking each other:

- API contract (S ↔ P).** By hour 3, S writes the endpoint list with example JSON and deploys **mock versions**. P builds every screen against the mocks and never waits for S.
- Agent contract (AI ↔ S).** Every agent is a pure Python function: typed input in, typed output + run metadata out. AI tests agents from a command line on sample prospects, no database or server needed. S just calls `run_agent("fitment", input)`.

4. AI, Explained From Zero

4.1 Core concepts

Concept	What it actually is	Where you use it
LLM API call	Send text (messages) over HTTP, get text back. The model has no memory between calls - anything it should know must be in the message.	Every agent
System vs. user prompt	System sets rules/persona (“You are an SDR for Sarvam...”). User carries the specific task and data.	Campaign + agent prompt in system; prospect data in user
Tokens	Models read/bill in tokens (~¾ of a word). You pay for input and output tokens; every response reports both.	Cost tracking: log <code>tokens_in</code> / <code>tokens_out</code> on every call
Context window	Max text one call can hold. More context = more cost, slower, often worse answers.	Send a compact prospect summary, not raw scraped pages
Temperature	Randomness. 0-0.3 = consistent; 0.7 = more creative.	Low for classification/decisions; ~0.7 for writing emails
Structured output	Force the model to return JSON matching a schema, then validate in code. This is what lets code act on model decisions.	Every agent returns JSON, never free text
Validation + repair	Models sometimes return broken JSON. Validate; if it fails, ask for a fix once; if that fails too, fall back safely and escalate.	Required by the brief (“handle malformed output”)
Tool / function calling	Describe functions to the model (<code>book_meeting(time)</code>); it says “call this with these arguments”; your code runs it.	Voice agent booking a meeting
Embeddings	A model turns text into ~1,000 numbers. Similar meaning → nearby numbers, enabling search by meaning.	Knowledge-base search
Vector database	Stores embeddings and finds “the 5 chunks closest to this question.” pgvector adds this to Postgres.	<code>kb_chunks</code> table
Chunking	Splitting documents into ~300-500 word pieces before embedding, so a search returns a relevant paragraph, not a whole document.	KB ingestion
RAG (retrieval-augmented generation)	Search the KB first, paste the best chunks into the prompt, tell the model to use only those facts.	Personalisation, Conversation, Voice
Hallucination	The model confidently makes things up (fake prices, fake customers).	Prevented by RAG + a grounding check + escalation
Grounding check	A second, cheap LLM call asking “is every claim in this email supported by these sources?”	Before anything is sent to a prospect
Agent vs. workflow	A fully free “agent” loops and chooses its own tools - unpredictable. A workflow is fixed steps with LLM decisions at defined points.	Build workflows, not free agents - more reliable
Model routing	Cheap, fast models for simple tasks; a strong model only where quality matters.	Fitment/classification: small model; email writing: strong model
Evals / golden dataset	Inputs with known correct answers, run through the agent to measure accuracy.	Measurement & Optimisation rubric points
LLM-as-judge	A model grades another model’s output against a rubric.	Scoring email quality
MCP (Model Context Protocol)	A standard way to hand an AI a set of tools: each tool has a name, description, and input fields. The AI reads the descriptions and decides which tool to call.	DronaHQ MCP tool layer (Section 7)

Provider choice. Pick one provider offering an OpenAI-compatible API, a small model, a strong model, and an embedding model. Keep model names in environment variables (`MODEL_SMALL` , `MODEL_STRONG` , `EMBED_MODEL`).

4.2 The LLM wrapper (build this first, hour 1)

Every agent goes through this wrapper - never call the SDK directly elsewhere.

```
# backend/ai/llm.py
import os, time, json
from openai import OpenAI
from pydantic import BaseModel, ValidationError

client = OpenAI(base_url=os.getenv("LLM_BASE_URL"), api_key=os.getenv("LLM_API_KEY"))
MODELS = {"small": os.getenv("MODEL_SMALL"), "strong": os.getenv("MODEL_STRONG")}

class AgentOutputError(Exception): pass

def strip_fences(t: str) -> str:
    return t.replace("`json", "").replace("`", "").strip()

def call_structured(system: str, user: str, schema: type[BaseModel],
                    tier: str = "small", temperature: float = 0.2):
    """Returns (validated_object, meta). meta goes into agent_runs for cost/debugging."""
    msg = f"{user}\n\nReturn ONLY JSON matching this schema:\n{json.dumps(schema.model_json_schema())}"
    for attempt in range(2):
        t0 = time.time()
        r = client.chat.completions.create(
            model=MODELS[tier], temperature=temperature,
            response_format={"type": "json_object"},
            messages=[{"role": "system", "content": system},
                     {"role": "user", "content": msg}],
            timeout=45)
        text = r.choices[0].message.content
        meta = {"model": MODELS[tier], "tokens_in": r.usage.prompt_tokens,
               "tokens_out": r.usage.completion_tokens,
               "latency_ms": int((time.time() - t0) * 1000), "attempts": attempt + 1}
        try:
            return schema.model_validate_json(strip_fences(text)), meta
        except ValidationError as e:
            msg += f"\n\nYour last answer was invalid: {e.errors()[0][2]}. Return corrected JSON only."
    raise AgentOutputError("invalid JSON after repair")
```

Cost = `tokens_in * input price + tokens_out * output price`. Keep a small price table in config.

4.3 The knowledge base (RAG)

Ingestion (run once, and again whenever documents change):

1. Read each Markdown file from `/knowledge/<campaign-or-global>/<doc_type>/`.
2. Split into chunks of ~400 words, overlapping by ~50 words.
3. Embed each chunk.
4. Insert into `kb_chunks` with `campaign_id` (null = shared by all campaigns), `doc_type`, `source`.

Retrieval:

```
-- in Postgres with pgvector; :q is the embedding of the search question
select id, content, source, doc_type
from kb_chunks
where (campaign_id = :cid or campaign_id is null)
   and doc_type = any(:types)      -- e.g. {'case_study', 'example_email'}
order by embedding <=> :q           -- <=> = cosine distance
limit 5;
```

The `campaign_id` filter gives each campaign its own knowledge (isolation, as required) while still sharing global documents.

Build a short search query rather than embedding the whole prospect record - e.g. "Indian private bank, collections calls, Hindi and Tamil customers, data residency concerns." Save the retrieved chunk IDs on the agent run so the UI can show sources.

4.4 The seven agents

Every agent: **load the active prompt version** → **retrieve context if needed** → **call** `call_structured` → **validate** → **return output + meta**.

Agent	Triggered when	Input	Retrieves from KB	Model	Output
Research	Prospect discovered	Name, company, domain, role	none - web search + company site	small	<code>company_summary, industry, size, recent_news[], likely_pains[], languages_served[], facts[{claim, source_url}]</code>
ICP Fitment	Research done	Research output + campaign ICP	ICP definition	small	<code>verdict: fit/no_fit/review, score 0-100, criteria_met[], criteria_missed[], reason</code>
Outreach Strategy (the brain)	Fitment passes; any reply/call/timer	Prospect summary + full cross-channel timeline + channel policy	playbook	small/strong	<code>action: send/call/wait/stop/escalate, channel, send_after_hours, intent, reason</code>
Personalisation	Strategy says send	Research facts + intent + channel + rep signature	case studies, example emails	strong (~0.7)	<code>subject, body, facts_used[], kb_chunks_used[]</code>
Conversation	Inbound reply	Reply + thread history	objection handling	small	<code>intent: interested/question/objection/not_now/unsubscribe/wrong_person/ooo, sentiment, needs_human, suggested_reply, referral_name?</code>
Voice SDR	Strategy says call	Prospect summary + goal	voice script, objections	voice platform	<code>transcript, outcome, qualified, meeting_time?, objections[]</code>
Follow-up	Timer fires, no reply	Timeline + touches so far	playbook	small	<code>follow_up: bool, channel, angle, wait_hours, reason</code>

The Strategy agent is the “one SDR brain.” Its prompt always includes the compact timeline:

```
[Day 0] LinkedIn: connection request sent - accepted Day 1
[Day 2] Email #1 sent (case study: Indian insurer) - opened 2x, no reply
[Day 5] Email #2 sent - no reply
Channel policy: linkedin, email, call(after engagement only). Touches this week: 2/3.
```

Rules such as “never repeat a channel ignored twice in a row” and “call only after engagement signals” are what make the system feel like one SDR moving across channels, not separate bots.

Grounding check, before any send:

```
SYSTEM: You are a strict fact-checker.
USER: MESSAGE: {draft} SOURCES: {facts + chunks}
List every factual claim in MESSAGE. For each, say supported/unsupported with the source id.
Return {"claims":[...], "all_supported": bool, "risky_topics": ["pricing"|"legal"|"commitment"...]}
```

If `all_supported` is false, regenerate once; if still false, send to the approval queue. If `risky_topics` includes pricing, escalate to a human. The UI shows the result as a “Grounded ✓” badge.

4.5 Voice agent

Use **Vapi** or **Retell** rather than building a custom voice stack - they handle real-time speech-to-text, LLM response, text-to-speech, and interruption handling.

- Create an assistant with a system prompt: goal, three qualifying questions, objection answers, “if asked about pricing, say a specialist will follow up.” Pick voice + transcriber.
- Define a tool `book_meeting(datetime)` pointing to `/webhooks/voice/tool`.
- Backend starts the call via the platform’s API, passing prospect variables that fill prompt placeholders.
- On call end, the platform webhooks the transcript to your backend; a small-model call extracts `qualified, objections, next_step`, then wakes the Strategy agent.

Test on your own phones by Saturday night. Record one good call as a backup for the demo. Hindi/Tamil is a stretch - check language support on the platform first.

4.6 Agents hosted in DronaHQ

Build two agents in DronaHQ’s Agentic Platform: a **Conversation Agent** (reply classification, called from your backend) and a **Manager Copilot** - a chat panel in the control plane that answers questions like “Why was the Infosys prospect disqualified?” using tools that call your backend.

4.7 Evals

1. **Fitment eval** - ~30 labelled prospects per campaign; report accuracy per prompt version.
2. **Reply-intent eval** - ~40 labelled example replies; report accuracy.
3. **Email quality** - an LLM judge scores each email 1–5 on personalisation, accuracy, clarity, call to action.

One script (`python -m eval.run --agent fitment --version 3`) writes to `eval_results` . Prompt Studio shows scores next to each version, so a manager sees e.g. “v3: 91% vs v2: 78%” before activating.

5. Software

5.1 Stack

Piece	Choice
API	FastAPI (Python - AI's code plugs in directly)
Database	Supabase Postgres + pgvector
Background work	A worker process in the same repo running the scheduler loop and job queue (Postgres <code>SKIP LOCKED</code>)
Hosting	Render or Railway: one web service + one worker. Public URLs needed for webhooks - deploy in hour 1
Auth for DronaHQ → API	An API key in a header on every request; <code>created_by</code> from the DronaHQ user

5.2 API endpoints

- **Campaigns:** `GET /campaigns` , `POST /campaigns` , `GET/PATCH /campaigns/{id}` , `POST /campaigns/{id}/{activate|pause|resume|complete|archive|duplicate}` , `GET /campaigns/{id}/preflight` , `GET /campaigns/{id}/metrics` , `GET /campaigns/{id}/prospects?stage=`
- **Prospects:** `GET /prospects/{cp_id}/timeline` , `POST /prospects/import (CSV)`
- **Prompts:** `GET /campaigns/{id}/agents` , `PATCH /campaigns/{id}/agents/{agent}` , `GET/POST /campaigns/{id}/prompts/{agent}/versions` , `POST .../versions/{v}/activate`
- **Queues:** `GET /inbox` , `POST /approvals/{id}/{approve|reject|edit}` , `POST /conflicts/{id}/resolve`
- **Controls:** `GET/POST /controls` (kill switch, channel pauses)
- **Reps:** `GET/POST /reps` , `POST /reps/{id}/offboard` , `POST /reps/reassign`
- **Other:** `GET /activity/feed?since=` , `GET /compare?ids=`
- **Webhooks:** `/webhooks/{whatsapp|voice|voice/tool|calendar}`

5.3 The policy gate

The safety system, run **right before any external action**.

```
def check_gate(cp, action) -> tuple[bool, str]:
    c = cp.campaign
    if controls.kill_switch:
        return False, "Global kill switch is on"
    if c.status != "live":
        return False, f"Campaign is {c.status}"
    if c.agent(action.agent).paused:
        return False, f"{action.agent} agent paused"
    if controls.channel_paused(action.channel) or c.channel_paused(action.channel):
        return False, f"{action.channel} paused"
    if suppression.contains(cp.prospect):
        return False, "Prospect on do-not-contact list"
    if touches_last_7d(cp.prospect) >= GLOBAL_CAP:
        return False, "Global contact limit reached"
    if owner_campaign(cp.prospect) not in (None, c.id):
        return False, "Another campaign owns this prospect"

    rep = cp.rep
    if not rep.in_working_hours(now()):
        return False, "Outside rep working hours"
    if rep.sent_today(action.channel) >= rep.daily_cap(action.channel):
        return False, "Rep daily limit reached"

    if needs_approval(c, action) and not action.approved:
        return False, "Waiting for approval"

    return True, "ok"
```

Blocked actions are stored as `held` , `rescheduled` , or `awaiting_approval` , and the reason string is shown **verbatim** in the UI. On resume, held actions go back to the Strategy agent, since the situation may have changed.

5.4 Scheduler loop (worker)

```
while True:
    due = db.claim_due(limit=10) # campaign_prospects where next_action_at <= now, row-locked
    for cp in due:
        try:
            step(cp) # stage-based: research → fitment → strategy → execute
        except Exception as e:
            log_failure(cp, e); cp.next_action_at = now() + backoff(cp.failures)
    poll_inboxes_every(30) # email replies
    sleep(3)
```

Idempotency. Every outbound touch has a unique key (`cp_id + step_number`), checked before sending, so a crash or retry never double-sends.

Demo speed. A per-campaign multiplier converts waits from hours to seconds (e.g. “1 day = 60 seconds”), shown as a labelled badge in the UI.

5.5 Channels

All channels share one interface: `send(touch) -> provider_id` plus inbound handlers, with `mode = live | sandbox`.

- **Email.** A fresh Gmail account with an app password. Send via SMTP, read replies via IMAP polling. Set `Message-ID / In-Reply-To / References` for threading. Use plus-addressing (`sarvamsdr+cmpB@gmail.com`) to route replies to a campaign automatically.
- **WhatsApp.** Twilio WhatsApp sandbox - avoids India’s SMS DLT registration problem. Inbound hits `/webhooks/whatsapp`.
- **Voice.** Vapi/Retell as in 4.5. Verify numbers on trial accounts; keep a browser web-call fallback.
- **LinkedIn.** A simulator: actions (view, connect, message) logged as touches; a “prospect simulator” panel lets Product accept a connection or reply live during the demo. State this honestly in the report.
- **Calendar.** Google Calendar API for free/busy + event creation, sent to the prospect’s email. Fallback: a Cal.com booking link with a webhook.
- **Discovery.** CSV import + the Research agent’s web search. Try Apollo’s API early; don’t depend on it.

5.6 Reliability and security

- Timeouts and retries on every external call.
 - Validate webhook signatures (Twilio, voice platform both sign theirs).
 - `.env.example` in the repo; real secrets only in the hosting platform’s environment variables.
 - A `/health` endpoint. Structured logs.
 - Every endpoint returns empty lists rather than errors on empty state.
-

6. Product: Control Plane and Everything Judges See

6.1 Visual language

- **Status colours:** Live = green with a pulsing dot; Paused = amber; Draft = grey; Completed = blue; needs attention = red badge.
- **One icon per channel**, used everywhere.
- **Plain-English reasons first** - e.g. “Held - campaign paused” or “Skipped call: prospect hasn’t engaged yet.” Raw JSON only behind “details.”
- **Auto-refresh** of the activity feed and counters every 5 seconds, so the system visibly feels alive.

6.2 Screens

1. **Mission Control.** Four campaign cards - status pill, ICP, mini funnel, today’s actions by channel, meetings count, a large Pause/Resume button. Alerts strip across the top. Global Kill Switch top-right (confirm by typing STOP). Live activity feed. Copilot chat as a side panel.
2. **Campaign page**, tabs: Overview (funnel Discovered→Opportunity, channel mix, outcomes, cost per qualified lead), Prospects (kanban by stage), Agents (7 cards, pause toggle, last run), Prompts, Knowledge (document list), Settings (ICP, channels, limits, reps, approval rules, demo speed).
3. **Prospect 360.** Research + source links + fit verdict on the left. One cross-channel timeline in the middle - each item shows agent, one-line reason, prompt version, Grounded badge. A “Next Best Action” card on the right with Approve/Edit/Skip.
4. **Inbox.** Approvals, escalations, conflicts in one queue, each resolvable in one click.
5. **Prompt Studio.** Version list, side-by-side diff, eval scores per version, Activate/Rollback, who changed what and when.
6. **Compare & Settings.** Campaign/variant metrics side by side. Reps with offboarding + reassignment. Suppression list. Integration status lights.

Pre-launch checklist, opened by pressing Activate: ICP set, prompts active, KB has ≥ 5 documents, rep assigned, channels connected, sample message previewed. The Live button unlocks only when everything is green.

6.3 Content Product creates

- **Knowledge base** (~25 Markdown files, from public Sarvam info): company overview; one product sheet each for Samvaad, the speech APIs, DocAgent; the startup program; 3-4 case-study-style write-ups from public announcements; an ICP definition per campaign; a sales playbook per campaign; an objection-handling guide; 3 good example emails per campaign; a voice script. **Never invent numbers** - if a fact isn’t public, leave it out.
 - **Prospects:** ~60 real companies (15 per campaign) with public info, where the *contact* email/phone belongs to teammates and friends acting as “prospect actors.”
 - **Golden datasets** for the AI evals (4.7).
 - **Seed history:** a few days of labelled past activity so dashboards aren’t empty, clearly marked as seeded.
-

7. Claude Code + DronaHQ MCP

7.1 What DronaHQ’s MCP support does

DronaHQ has an **MCP server builder**: it exposes the connectors you set up in DronaHQ (databases, APIs) as **MCP tools** an AI can call. Once created, you connect the server to Claude Code by pasting server details into the terminal or editing a config file.

MCP, in plain terms: a standard way to hand an AI a set of tools, each with a name, description, and input fields. The AI reads the descriptions, decides which tool to call, and gets a result back - the same idea as tool calling in Section 4.1, packaged so any AI client can plug into any tool server.

Verify in the first 3 hours. Connect the DronaHQ MCP server to Claude Code, run `/mcp`, list the tools. Then check: can it only call your connectors (likely), or can it also **create/edit DronaHQ apps and agents**? The plan works either way - if it can build apps, Product gets a speed boost (7.4).

7.2 Architecture: one shared tool layer

Build a **DronaHQ MCP server named** `sdr-tools` on top of two DronaHQ connectors: a **REST connector** to your FastAPI backend, and a **read-only Postgres connector** to Supabase (a database user restricted to SELECT).

These tools serve three consumers: the DronaHQ-hosted agents (Copilot, Conversation Agent) at runtime; Claude Code while building (inspecting live data, debugging); and a manager running the SDR by chat during the demo.

Tool	Type	Calls	Used by
<code>list_campaigns</code>	read	<code>GET /campaigns</code>	Copilot, Claude Code
<code>get_campaign_metrics(campaign_id)</code>	read	<code>GET /campaigns/{id}/metrics</code>	Copilot, Claude Code
<code>get_prospect_timeline(cp_id)</code>	read	<code>GET /prospects/{id}/timeline</code>	Conversation Agent, Copilot
<code>explain_action(touch_id)</code>	read	<code>agent_runs</code> joined with prompt version + KB chunks	Copilot (“why did the agent do this?”)
<code>search_knowledge(campaign_id, query, types)</code>	read	<code>POST /kb/search</code>	Conversation Agent, Copilot
<code>list_inbox</code>	read	<code>GET /inbox</code>	Copilot
<code>pause_campaign / resume_campaign</code>	write	<code>POST /campaigns/{id}/pause</code>	Copilot, manager via Claude
<code>approve_action(approval_id)</code>	write	<code>POST /approvals/{id}/approve</code>	manager via Claude

Safety rules for write tools:

- Write tools **only call your backend endpoints**, never the database directly - so the policy gate, permission checks, and audit log still apply.
- Every call records who made it (`actor = "mcp:copilot"` or `"mcp:claude-code"`); the Activity feed shows it, e.g. “Campaign paused by Copilot on request of Priya.”
- Keep **two environments**: staging (Claude Code builds/experiments here) and production (the demo) - two Supabase projects or schemas, each with its own MCP server, so Claude Code never touches demo data by accident.

One-line architecture for the report: *DronaHQ Apps (control plane) + DronaHQ Agents (Copilot, Conversation) + a DronaHQ MCP tool layer, all on top of your own FastAPI orchestration engine, policy gate, and channel services.* DronaHQ now sits in three places - UI, agents, tool layer - which is hard to call “a wrapper.”

7.3 Setting up the repo for Claude Code

`CLAUDE.md` **at the repo root** (read automatically every session - keeps three parallel sessions consistent):

```
# Autonomous SDR - Sarvam AI demo (Inter Guild Buildathon)

## What this is
Multi-campaign autonomous SDR. FastAPI backend + worker, Supabase Postgres/pgvector,
DronaHQ control plane + DronaHQ agents + DronaHQ MCP tool layer.
Read docs/architecture.md and docs/api_contract.md before changing interfaces.

## Folder ownership (don't edit another owner's folder without asking)
- backend/api, backend/orchestrator, backend/channels, backend/db -> Software
- backend/agents, backend/ai, backend/rag, backend/eval -> AI
- dronahq/, knowledge/, data/, docs/ -> Product

## Non-negotiable rules
- Every external action goes through orchestrator/policy_gate.check_gate(). Never bypass.
- Every LLM call goes through ai/llm.py call_structured(). Never call the SDK directly.
- Agents are pure functions: typed Pydantic input -> (output, meta). No DB access inside agents.
- Never invent Sarvam facts, prices, customers. Knowledge comes only from knowledge/.
- No secrets in code, CLAUDE.md, or .mcp.json. Use env vars; update .env.example.
- Empty states return [] not errors. Wrap external calls with timeout + retry.
- Delete unused code. No commented-out blocks, no duplicate helpers.

## Commands
- Run API: uvicorn backend.api.main:app --reload
- Run worker: python -m backend.orchestrator.worker
- Tests: pytest -q Lint: ruff check . --fix
- Evals: python -m backend.eval.run --agent fitment --version latest
- Ingest KB: python -m backend.rag.ingest

## Conventions
Python 3.11, type hints, Pydantic v2, snake_case, one module per agent,
commit messages: "area: what changed" (e.g. "agents: add grounding check").
```

Add shorter `CLAUDE.md` files inside `backend/agents/` (the agent pattern and output schemas) and `backend/channels/` (adapter interface, sandbox mode).

`.mcp.json` - connecting DronaHQ, committed without secrets:

```
{
  "mcpServers": {
    "sdr-tools-staging": {
      "type": "http",
      "url": "${DRONAHQ_MCP_STAGING_URL}",
      "headers": { "Authorization": "Bearer ${DRONAHQ_MCP_TOKEN}" }
    }
  }
}
```

Each person sets these variables locally. Copy the exact URL/auth format from DronaHQ’s MCP server page. Deliberately leave the **production MCP server out** of this file.

Custom commands (`.claude/commands/*.md`):

Command	What it tells Claude Code to do
<code>/new-agent <name></code>	Create an agent module: Pydantic input/output, a default prompt, a CLI test on samples, a unit test with a mocked LLM
<code>/new-endpoint <spec></code>	Add the endpoint to <code>docs/api_contract.md</code> first, then implement with validation, empty state, and a test
<code>/check</code>	Run ruff + pytest, fix failures, summarise the change
<code>/explain <file></code>	Explain the file line by line for someone new to AI/backend (the AI person’s tutor)
<code>/cleanup</code>	Find dead code, unused imports, duplicate helpers, TODOs, hardcoded secrets; propose deletions (run Sunday)
<code>/why <prospect></code>	Use <code>sdr-tools-staging</code> to fetch the timeline and agent runs, and explain what happened

Working practices: specify → plan → build → check, reading every diff before committing; small tasks with `/clear` between unrelated ones; paste real docs URLs rather than relying on memory for Twilio, the voice platform, Gmail, DronaHQ; separate branches/worktrees per person, merging to `main` every 2-3 hours; commits from all three people, small and often; be mindful of usage limits - front-load heavy generation, use the stronger model for planning/hard debugging.

7.4 How each role uses it

Product. If the DronaHQ MCP can build apps, describe each screen and have Claude Code create it, then fine-tune by hand. If not, Claude Code still writes DronaHQ’s query/transformer code, formulas, and custom-component HTML/CSS - you paste it in and own the look. Use it to draft the ~25 KB files from Sarvam’s public pages with source URLs, then **fact-check every one yourself**. Use it to draft prospect data and golden-dataset replies, but **you set the labels**. In the last 5 hours, have it draft the README’s file-structure section, the architecture

diagram (Mermaid), and the report's "what works/partial/skipped" section - you edit for honesty.

AI. Use `/explain` before building each new concept - the rule is **you must be able to explain every agent to a judge without help**. Let Claude Code scaffold modules, schemas, tests, and the eval runner via `/new-agent`; the prompts themselves are your job - run on 10 samples, read outputs, tweak, rerun. Build the Copilot and Conversation Agent in DronaHQ's Agentic Platform, attaching `sdr-tools` as their tools; write each tool description carefully, since the agent picks tools based on that text.

Software. Scaffold FastAPI, migrations, the job runner, and the policy gate in the first 6 hours using `/new-endpoint` - but **write the policy-gate tests yourself first** (paused campaign blocks send; kill switch blocks everything; suppressed prospect blocks every campaign), then let Claude Code implement until they pass. One session per channel adapter, real docs pasted in, sandbox mode first. Set up both DronaHQ connectors and both MCP servers (staging + production), including the read-only DB user. When debugging, ask Claude Code to use `sdr-tools-staging` to inspect agent runs and logs together.

8. Updated Timeline (40 effective hours)

Roughly 11 hours of staggered sleep/meals within the 51-hour window. With Claude Code, build phases shrink by about a third - put the saved time into reliability, evals, UI clarity, and demo rehearsal, not more features.

Window	Product	AI	Software	Checkpoint
Fri 21:00-24:00	DronaHQ tests + test what MCP can do; start KB with Claude Code	llm.py, /explain sessions, test DronaHQ agent + one voice call	Repo, CLAUDE.md, commands, .mcp.json, schema, deploy, API contract + mocks	CP0: every risky assumption confirmed or replaced; everything deploys
Sat 00:00-04:00	Mission Control + campaign page (mock data)	Research + Fitment agents from the CLI	CRUD, lifecycle, policy-gate tests then implementation	
Sleep, staggered				
Sat 09:00-14:00	KB fact-check, prospects, Prospect 360	Retrieval, Strategy, Personalisation, grounding check	Scheduler, email adapter, agent_runs logging, both DronaHQ connectors + MCP servers	CP1+CP2: full email loop live on the deployed site by 14:00
Sat 14:00-22:00	Inbox, Prompt Studio, Settings, pre-launch checklist	Conversation Agent on DronaHQ w/ MCP tools, Follow-up, voice agent	WhatsApp, voice webhooks, calendar, LinkedIn simulator, conflicts, 4 pause levels	CP3: 4 campaigns concurrently by 22:00
Sat 22:00-Sun 02:00	Compare page, Copilot panel	Copilot on DronaHQ + tool descriptions; evals	Metrics, seed data, production MCP server (write tools + audit)	
Sleep, staggered				
Sun 09:00-14:00	Visual polish, full UX walkthrough	Tune prompts from eval failures; Hindi/Tamil call	Reliability pass: timeouts, retries, empty states; /cleanup	Quality checkpoint
Sun 14:00-17:00	Rehearse demo x2, record backup video	Rehearse Copilot + voice parts	Fix bugs only; switch Claude Code to read-only MCP tools	Feature freeze at 17:00
Sun 17:00-22:30	Report, deck	AI section of report	README, diagram, final /cleanup	Submit by 22:30

If you fall behind, cut in this order: calendar API (use a booking link) → Copilot → B-Variant/Compare → LinkedIn-simulator polish → Hindi/Tamil voice. **Never cut:** the email loop, the policy gate and pauses, three-plus concurrent campaigns, grounding, Prospect 360.

9. Risks and Mitigations

Risk	Mitigation
DronaHQ has unknown limits (custom components, API calls, publishing/sharing)	Verify in hour 0-3; keep all logic in your backend so DronaHQ only calls your API
Judges can't open the deployed app	Test publishing on hour one; demo credentials in the README/report; test from a fresh browser
Twilio trial restrictions, India SMS rules	Use the WhatsApp sandbox, verified numbers, browser voice-call fallback
Voice latency / awkward calls	Short script, one qualification goal, rehearsed, with a recorded backup call
Apollo credits/rate limits	Cache aggressively; CSV + web-search enrichment fallback; pre-enrich demo prospects
Malformed LLM JSON	Schema validation, one repair retry, safe fallback, escalation record - never crash
Hallucinated claims in emails	Mandatory retrieval before writing; grounding judge; unsupported claims go to approval
Duplicate sends from concurrent workers	<code>SKIP LOCKED</code> on jobs, idempotency key per touch, gate re-check right before sending
Reply matched to wrong prospect/campaign	Match by thread ID, fallback to sender address; plus-addressing/hidden header per campaign
Follow-ups take days, demo takes minutes	The demo-speed multiplier
Dashboards look empty	Seed clearly-labelled historical data alongside the live loop
Secrets leak into the repo	<code>.env.example</code> only, gitignore from commit 1, secrets in the hosting platform's env vars
Three Claude Code sessions produce inconsistent code	Shared <code>CLAUDE.md</code> , API contract, folder ownership, merges every 2-3 hours
Code volume grows faster than anyone understands it	Read every diff, run <code>/explain</code> , run <code>/cleanup</code> Sunday
Claude Code invents an outdated API for Twilio/voice/DronaHQ	Paste real docs URLs; test against the real sandbox immediately
An MCP write tool changes demo data by accident	Separate staging/production MCP servers; production write tools reachable only via Copilot; read-only after freeze
DronaHQ's MCP tools are slow or limited	The REST API is the source of truth; UI and orchestrator work without MCP - it's additive
AI person can't answer judges' questions	The <code>/explain</code> routine + a 20-minute mock Q&A Sunday afternoon
Hitting usage limits on Sunday	Front-load heavy generation Saturday; Sunday is mostly review and fixes
Scope creep	Anything not required for a milestone waits until after Milestone 2 (Section 8)

10. Repository Structure

```
/backend
  /api          routes: campaigns, prospects, prompts, controls, webhooks
  /agents       one module per agent + schemas + prompts defaults
  /orchestrator scheduler, job runner, policy_gate.py
  /channels     email, whatsapp, voice, linkedin_sim (common interface)
  /rag          ingestion, retrieval, grounding check
  /eval        golden sets, runners, llm_judge
  /db          models, migrations, seed
/dronahq       exported app/agent configs + screenshots
/knowledge     source docs per campaign
/docs          architecture diagram, report, deck
CLAUDE.md     .mcp.json  README.md  .env.example
```

Commit small and often from every teammate - judges look at commit history.

11. Demo Script (5-7 minutes)

1. **Mission Control.** Three-plus campaigns Live, each with a different ICP; the activity ticker is moving.
2. **Launch a new campaign.** Duplicate an existing one into a variant, change one prompt, walk through the pre-launch checklist.
3. **Prospect 360.** Open a Voice AI founder - research with sources, the fit verdict, a grounded email with its sources. Approve it.
4. **A teammate replies live** (“Interesting, but how does pricing work? Call me.”) - the Conversation Agent classifies it, Strategy chooses a voice call.
5. **A phone rings on stage** - the call qualifies the prospect and books a meeting; the invite appears on the timeline.
6. **Pause a campaign** (e.g. India BFSI). Its pending action shows “Held.” The other campaigns keep running.
7. **Show a conflict** - the same CTO in two campaigns; resolve it from the Inbox.
8. **Prompt Studio.** Diff two versions, show the eval-score change, roll back.
9. **Ask the Copilot** - “Why was this BFSI prospect held, and pause the US SaaS campaign” - show the MCP tool call, the policy gate holding the action, and the audit entry in the Activity feed.
10. **Global Kill Switch.** Everything stops. Show the Compare page: cost per qualified lead.

The single most important rule: have a working version on the deployed URL by Saturday afternoon, and only add to it from there.